



Labsheet 1: ARM Assembly Introduction

Name:

Roll Number:

Part 1 – Quick Overview of the ARM

Before looking at the ARM processor in detail, we provide a very brief overview. The ARM processor is classified as a 32-bit RISC (Reduced Instruction Set Computer) with a three-operand register-to-register instruction set. This is simply a way of saying that computer operations involve three operands in registers, such as: `ADD R3, R2, R1`. There are a few instructions that have two operands and some that have four, but that doesn't change the overall classification.

Data Transfer Instructions

To get data into and out of registers (transfers between memory and registers), there are two special instructions called **load** and **store**.

- **Load** transfers data from memory to a register.
- **Store** transfers data from a register to memory.

These instructions have the forms: `LDR R0, [R1]` and `STR R0, [R1]`

As we have seen, these instructions use **register indirect (i.e., pointer-based) addressing**. The location of the memory element to be accessed is held in a register, and the addressing mode is indicated by `[R1]`.

Address Setup Using ADR

The ARM uses a special instruction called **ADR (load register with an address)** that sets up a pointer in the first place. For example:

`ADR R0, List`; register R0 points at the list.

Later, we will explain why this is a special instruction.

Data Processing Instructions

An ARM instruction like: `SUB R3, R2, #4`

Subtracts the actual value **4** (remember that the literal is indicated by the `#` symbol) from the contents of register **R2** and puts the result in **R3**.

Data operations implemented by ARM processors write the **destination (result) operand first on the left**. We write the destination operand in bold font to remind you where the result goes.

23CSE304: Embedded Systems

Simple Example

Let's create a very simple example:

```
MOV R0, #2      ; Put 2 in register R0
MOV R1, #3      ; Put 3 in register R1
ADD R2, R1, R0   ; Add R0 to R1 and put the result in R2
MOV R4, #10     ; Put 10 in R4 (this is where we are going to store the result)
STR R2, [R4]     ; Store R2 in memory location 10
```

The Instruction

Computer instructions are executed sequentially, one by one in turn, unless a special instruction deliberately changes the **flow of control** or unless an event called an **exception (interrupt)** takes place.

The structure of instructions varies from machine to machine. The format of an instruction running on an Intel processor is different from the format of an instruction running on a 68K or an ARM (even though the instructions might do the same thing). Instructions are classified by what they do and by the number of operands they take. The three basic instruction types are:

- **Data movement** that copies data from one location to another
- **Data processing** that operates on data
- **Flow control** that modifies the order in which instructions are executed

Instruction formats can take zero, one, two, three, or even four operands. Consider the following examples of instructions with zero to three operands. In these examples operands **P**, **Q**, and **R** may be memory locations or registers.

Operands	Instruction	Effect
Three	ADD P , Q , R	Add R to Q and put the result in P
Two	ADD P , Q	Add P to Q and put the result in P
One	ADD P	Add P to an accumulator
Zero	ADD	Add the top two items on the stack and push the result

A three-address computer instruction can be written as:

operation destination, source1, source2

where **operation** defines the nature of the instruction, **source1** is the location of the first operand, **source2** is the location of the second operand, and **destination** is the location of the result.

The instruction ADD R3, R1, R2 adds R1 and R2 to get R3.

Part 2 – Understanding and writing an ARM Assembly program

Familiarization of ARM Simulator

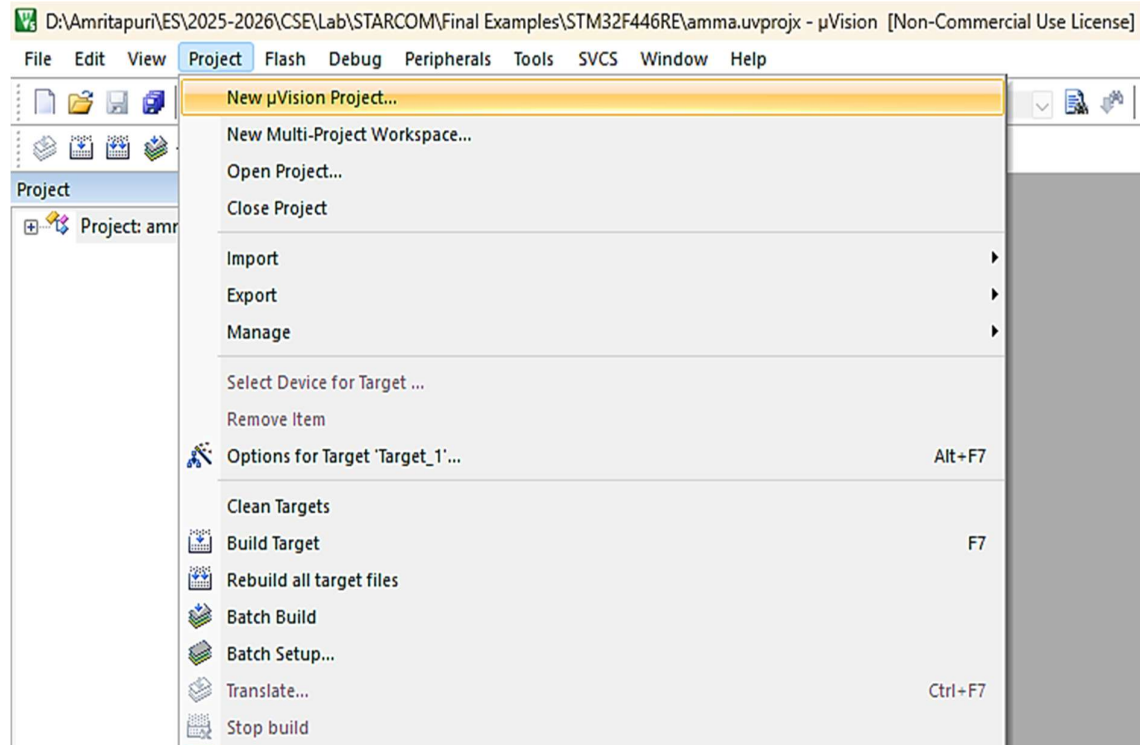
Keil MicroVision is an integrated development environment (IDE). Keil µVision 5 is software used for writing programs, compiling and testing the program without real hardware i.e. simulation. Following section describes the steps involved in creating a project, building it and debugging the same.

23CSE304: Embedded Systems

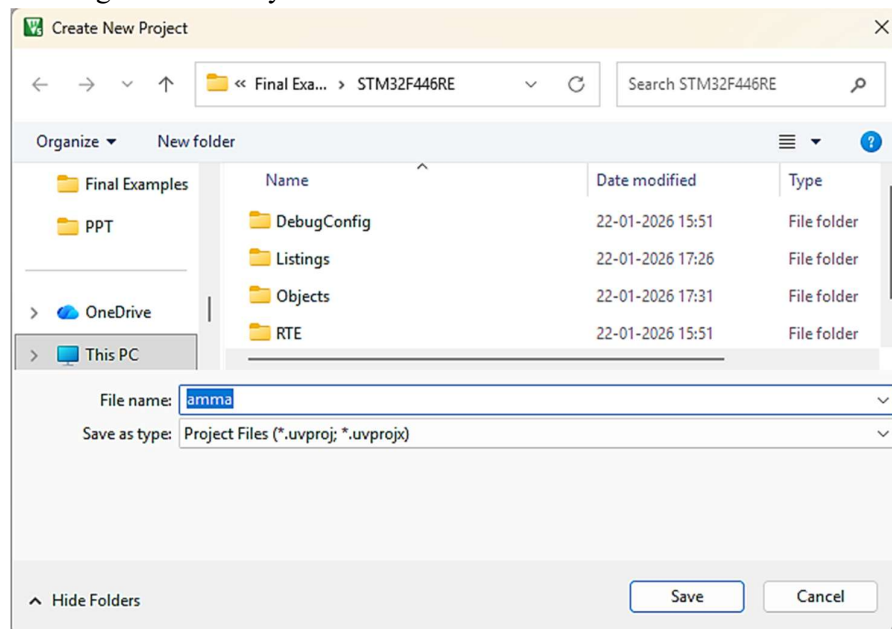
Creating an Assembly Project in Keil μ Vision 5

Step 1. Launch **Keil μ Vision 5** from your Start menu or desktop icon .

Step 2. Go to **Project** tab $\rightarrow$ **New μ Vision Project**.



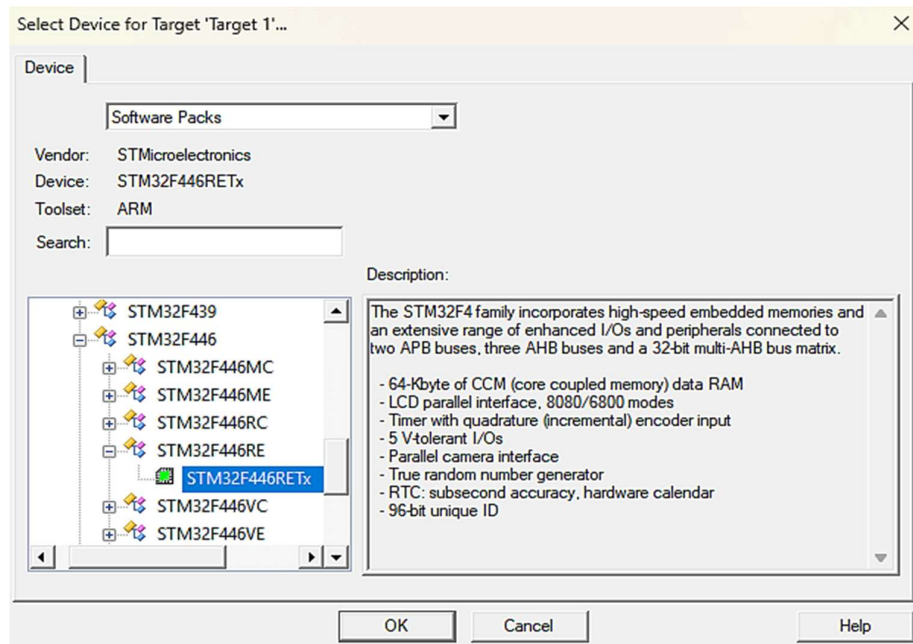
Step 3. Create a **New Folder** and give name to your project. Ensure that you create all the folders under a folder having the name as your roll number. Click **Save**.



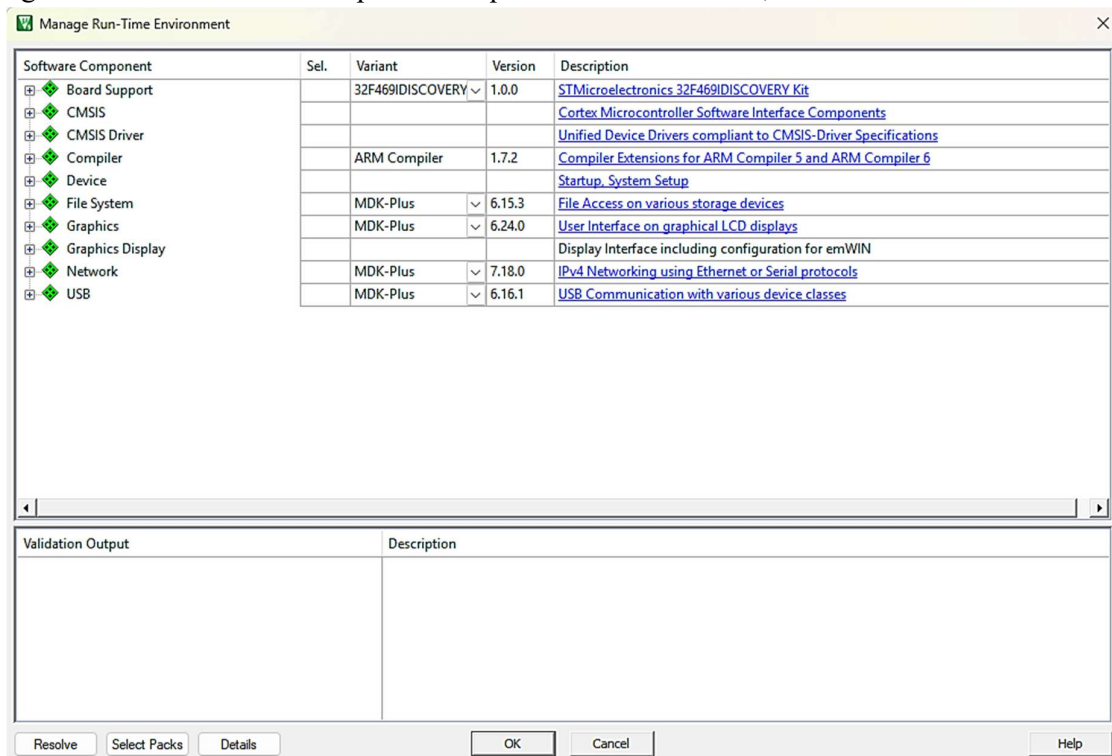
NB: - It is better to always use New Folder for every new Project. Try to give sensible name to the folder and the project. The project name will be stored with .uvproj extension by default.

23CSE304: Embedded Systems

Step 4. From the data base of vendors, select the vendor and the device model. We can select **STMicroelectronics**→**STM32F4series**→**STM32F446RE**→**STM32F446RETx** to work and understand the basics of Assembly language and the working of the Instruction Set, then click **OK**.

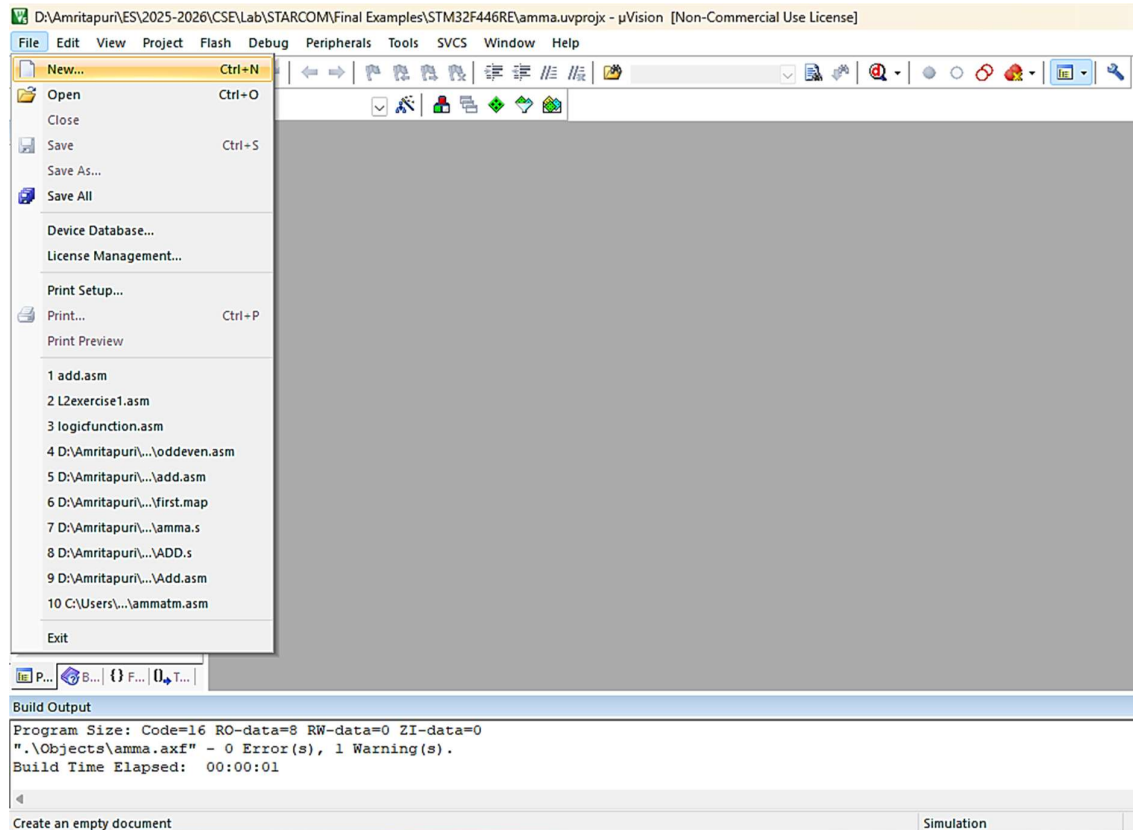


Step 5: The **Manage Run-Time Environment** window in Keil MDK is used to select and configure essential software components required for STM32F466, click **OK**.

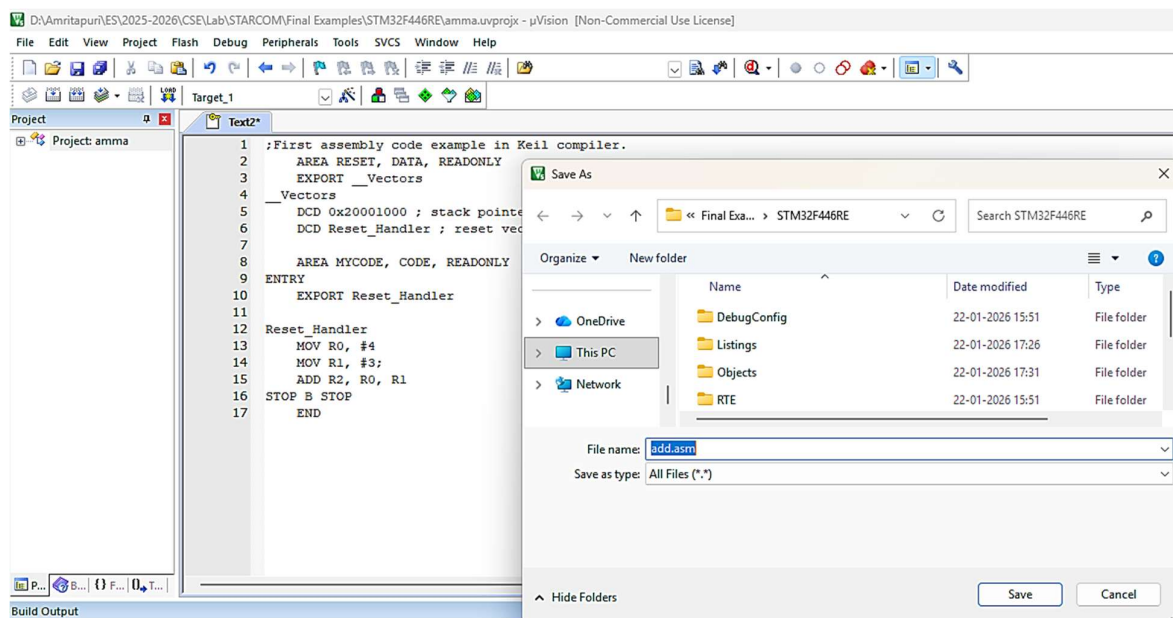


Step 5. Create a New File: Go to **File** → **New** to open a new text editor window. Write your program code in the new white workspace created.

23CSE304: Embedded Systems

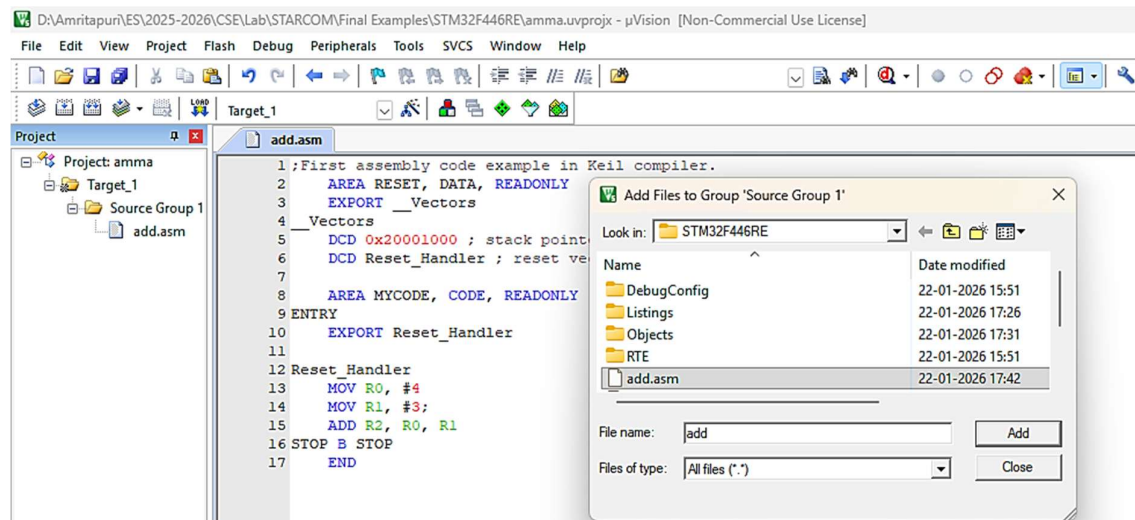
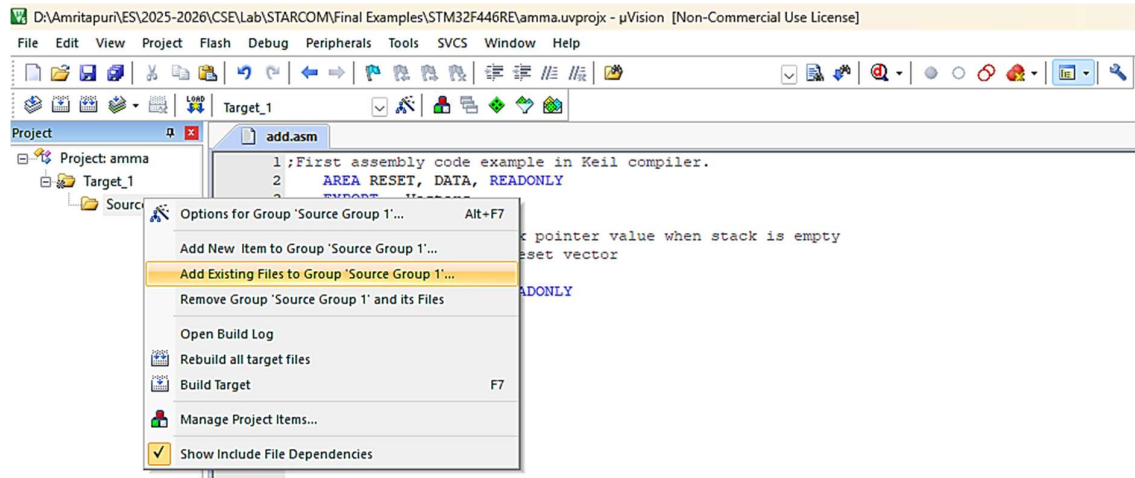


Step 6. After writing the codes go to **File** → **Save** (or **Ctrl+S**). Save the file with **.c** extension if the program is written in C or with **.asm** if the program is in assembly. The file must be saved in the same project folder.

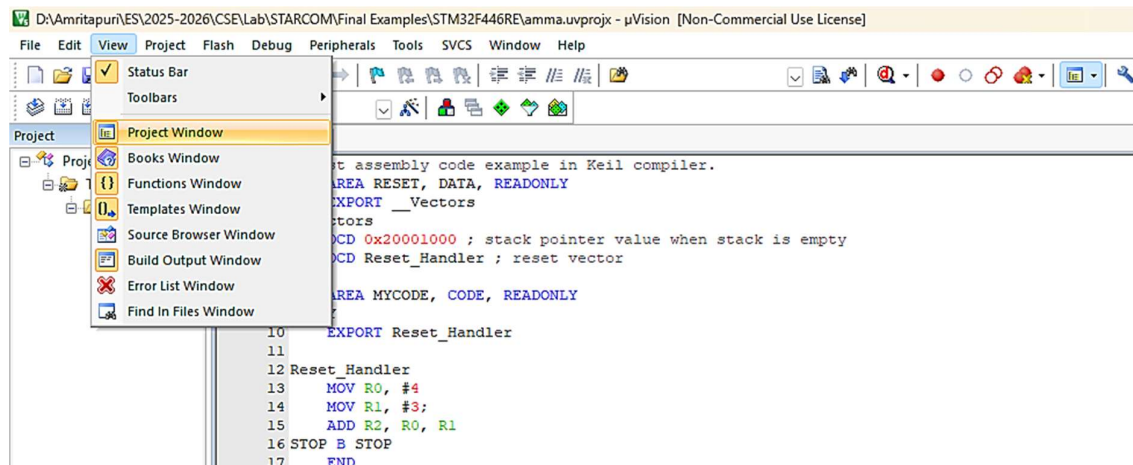


23CSE304: Embedded Systems

Step 8. Now add the file to your project. In the **Project** window, Right click on **Source Group** → Select **Add Existing Files to Group Source Group 1**. Select your saved .asm/.c file from folder and click **Add**, then **Close**.

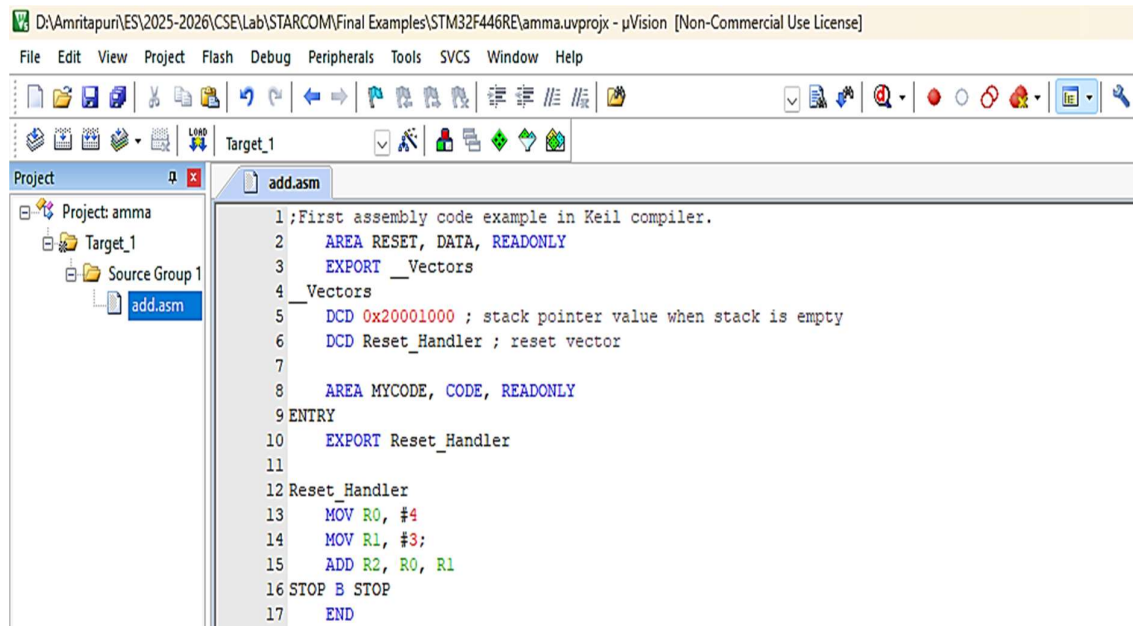


Note: - If Project Window is not there, go to **View** → **Project Window**



23CSE304: Embedded Systems

Note: - You may check the .asm file by clicking on the little plus sign at the left of **Source Group**.

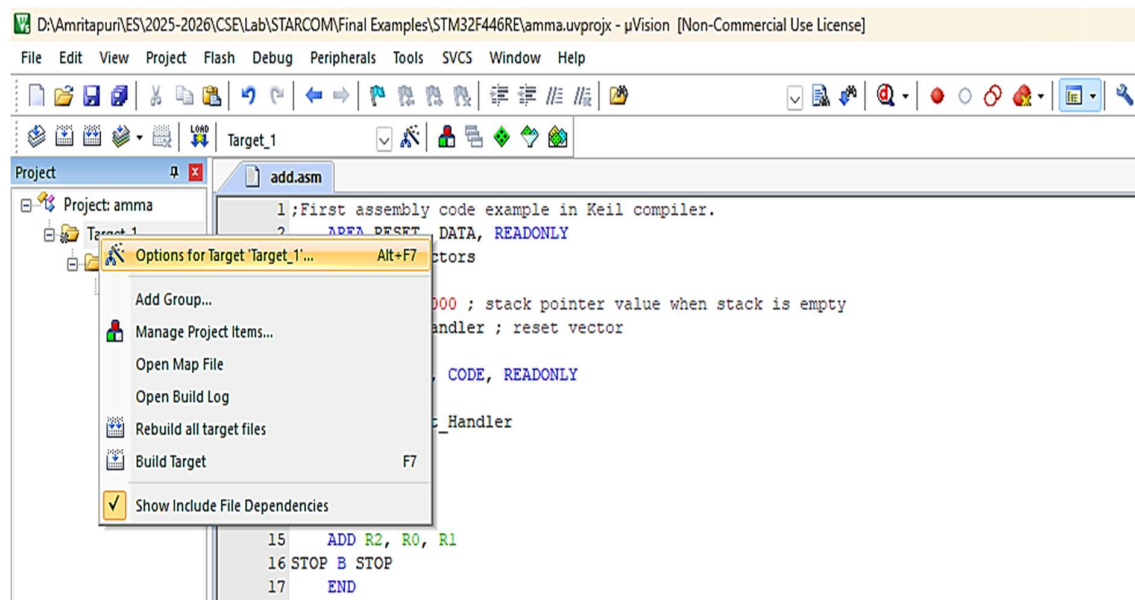


Note: -

AREA MYCODE, CODE, READONLY; ENTRY; END – all these 3 lines are assembler directives and are not part of the assembly language. The purpose of AREA directive is to name the region of memory where the program is located.

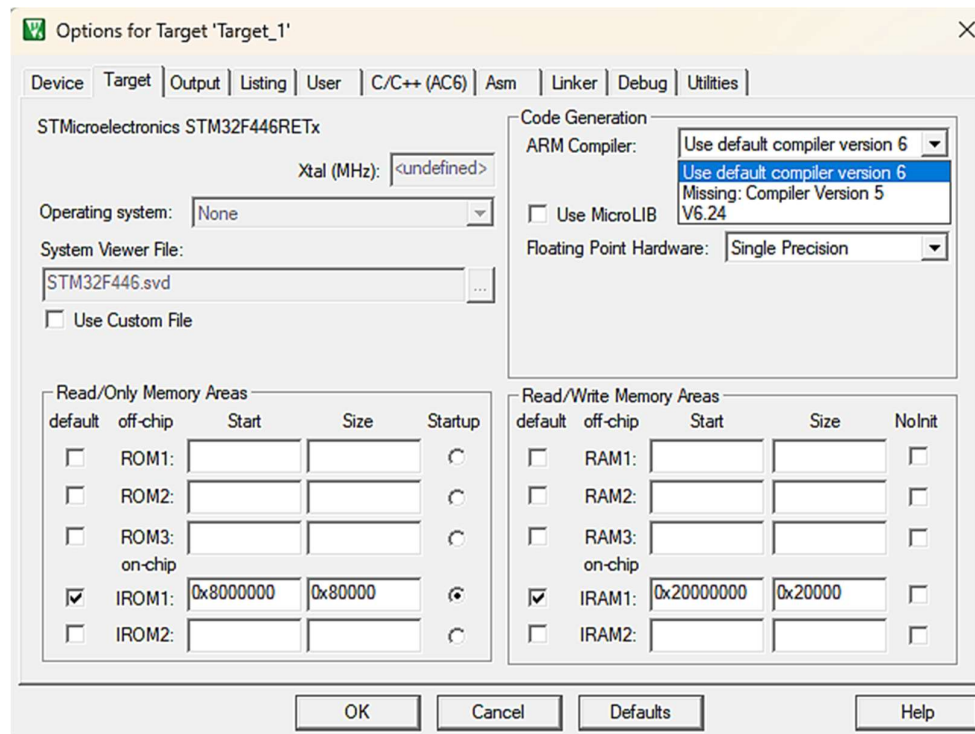
Building the Project

Configure Project Options: Right-click on your target name (e.g., Target 1) in the Project Window and select **Options for Target 'Target 1'**.

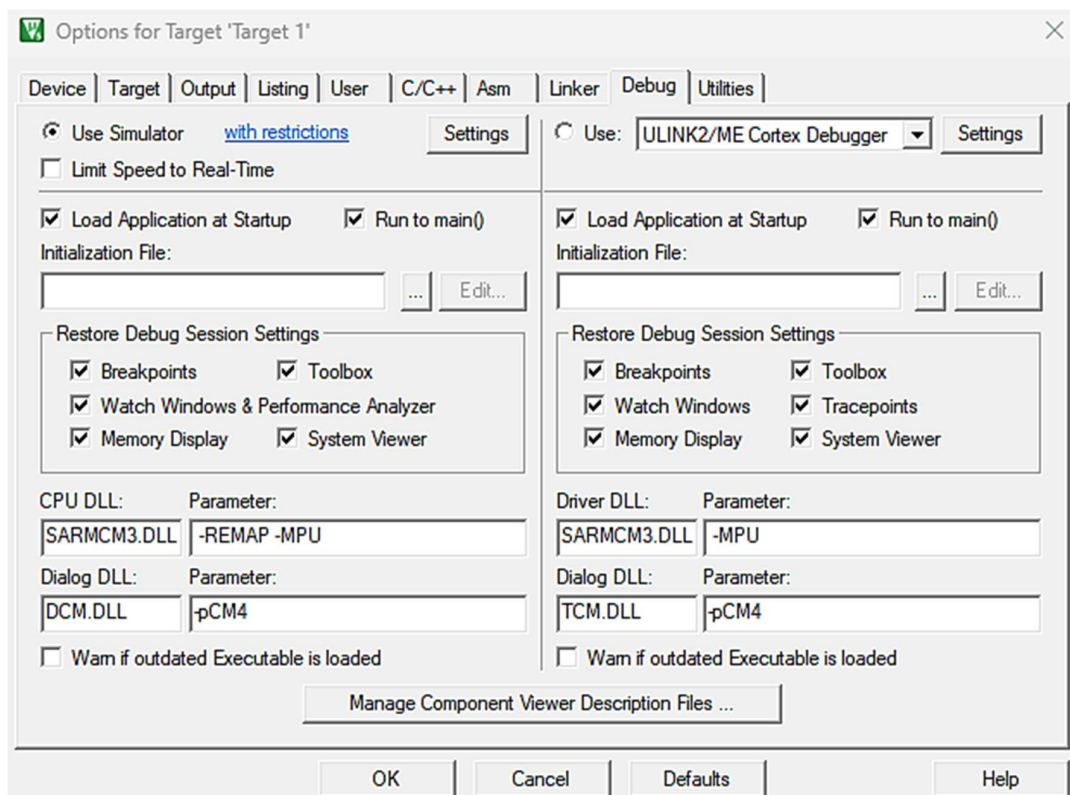


23CSE304: Embedded Systems

In the **Target** tab of the **Options for Target** window, select **Use default compiler version 6**.



Select the **Debug** tab and choose **Use Simulator** as the debug driver, then click **OK**.



23CSE304: Embedded Systems

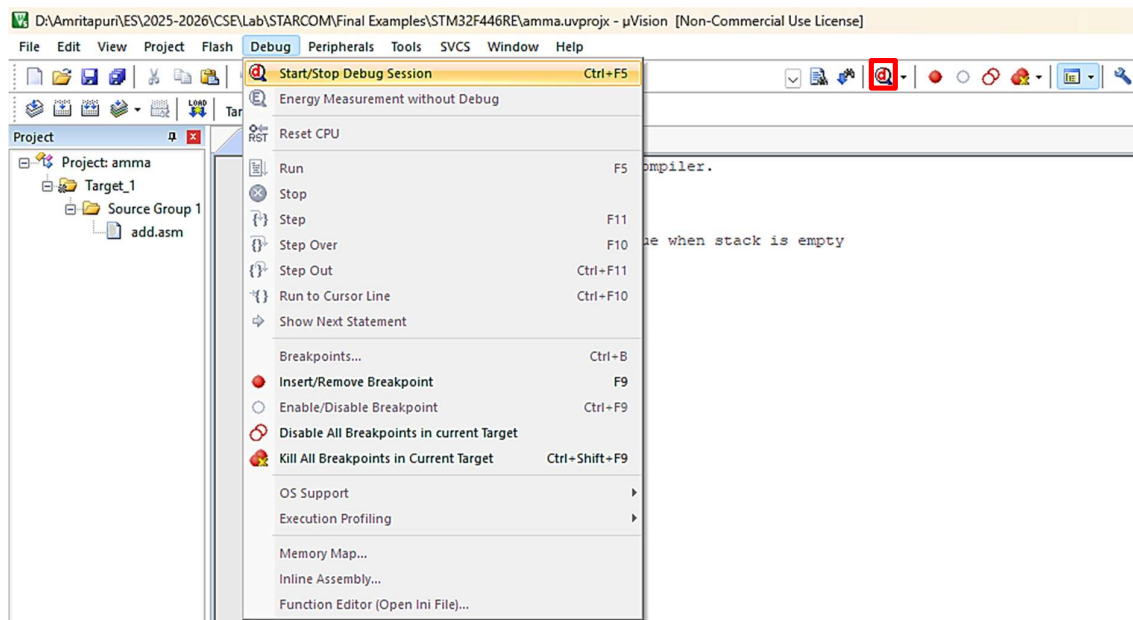
1. Goto **Projects** → **Build target** / **F7** or use the icon highlighted



Once you build your project, if there are errors in the code, it will be displayed at the Build output window along with line number. If there are no errors code will be compiled and status will be shown at the Build output window.

Debugging and Simulation

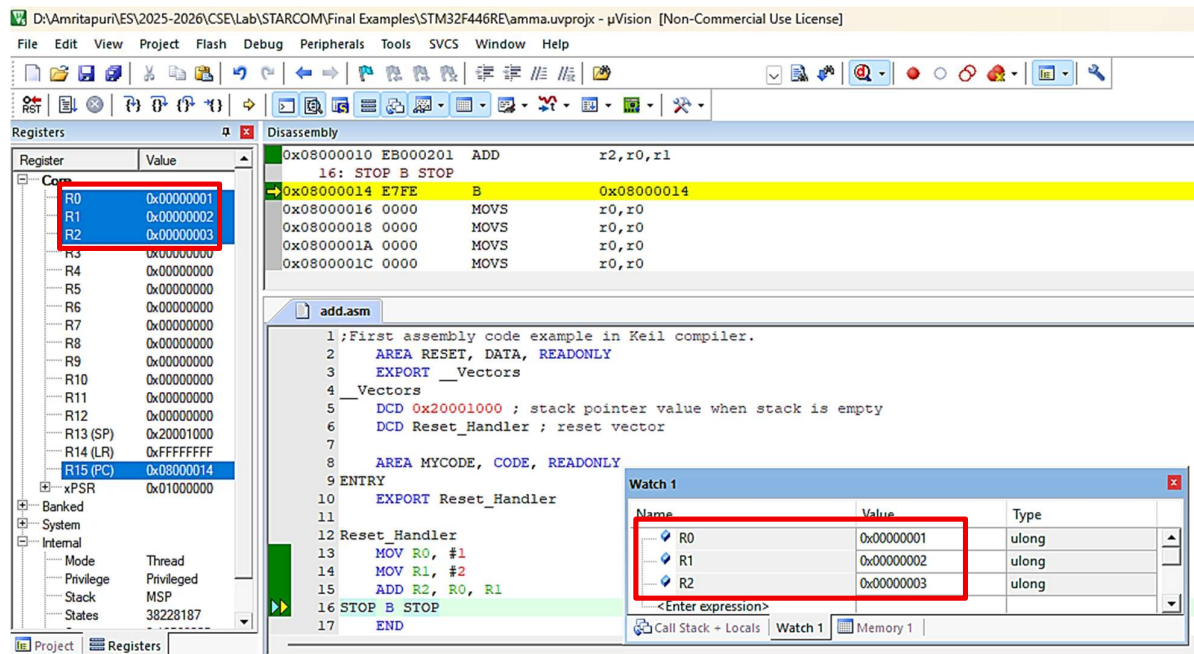
Go to **Debug** → **Start/Stop Debug Session** / **Ctrl+F5**.



Debugging windows:

There are some special windows provided by the Keil IDE, to help you to analyze the code and result through the Registers and Memory.

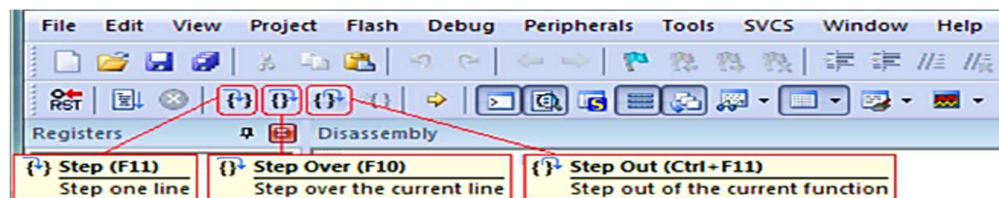
23CSE304: Embedded Systems



Register Window: The left most panel is called register window that displays the contents of all the ARM processor registers and allows you to modify their contents. A register can hold any 32-bit value. That value can be a (signed) int, an unsigned int, a pointer (memory address). All C variables map onto the registers and large data structures like arrays will be in a memory. But ARM arithmetic instructions only operate on registers, never directly on memory. Register window can be very handy for debugging. Go through the required registers and verify their contents.

Disassembly window: In disassembly window, you see the listing, which can contain both C or high-level language program and its corresponding assembly instructions. Current instruction is pointed with yellow arrows; break points are shown in the margin as red bullets.

To trace your program you are provided with Step Over button (or click on Step Over from the Debug menu). It executes the instructions of the program one after another.



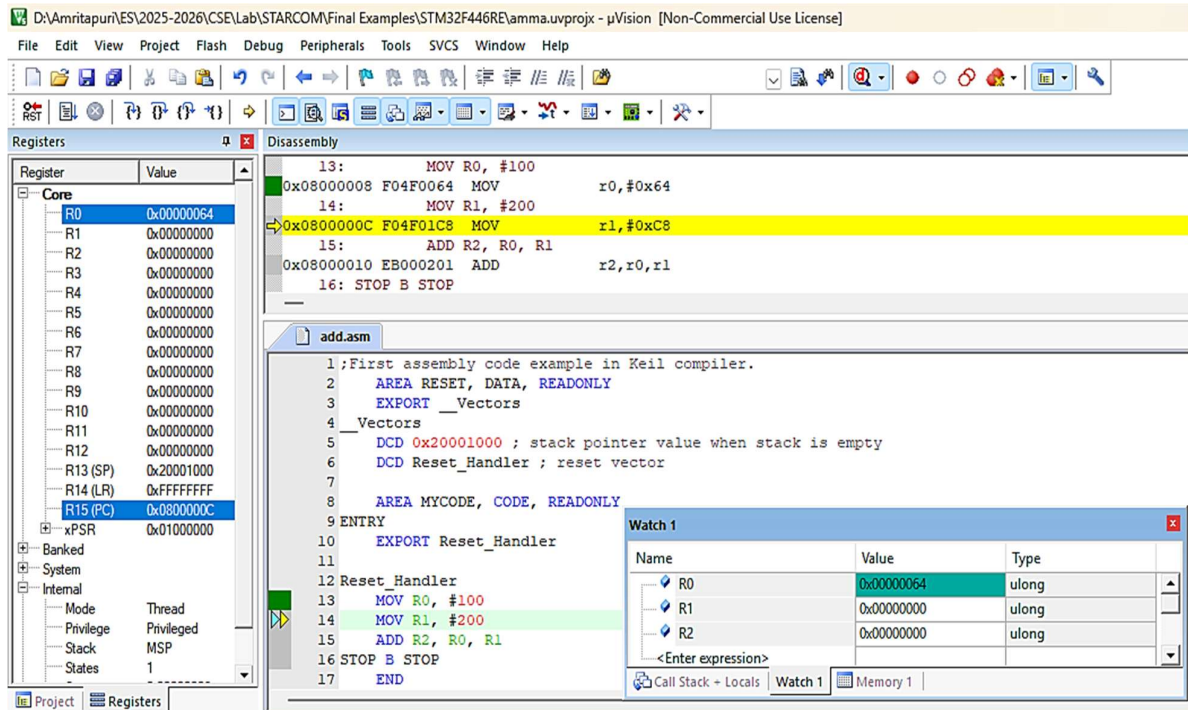
After the debugging is over, stop the debugging session by again selecting **Start/Stop Debug Session** from **Debug** menu.



23CSE304: Embedded Systems

Questions to Answer:

1. Based on the screenshot below, what is the current instruction that it is executing/debugging?
_____ is the instruction as specified in the add.asm file and it is in the address _____ as shown in the disassembly window and the current instruction address is specified in the register _____.



2. After the execution of the entire program once, R0 contains the value _____, which is the hexadecimal of the decimal _____. R1 contains the value _____, which is the hexadecimal of the decimal _____. R2 contains the value _____, which is the hexadecimal of the decimal _____.
3. Write your assembly program to add the contents of registers R1, R2, R3, R4 and R5 and save the result in R0. You can initialize the contents of R1 to R5 as 0x10, 0x2A, 0x3B, 0x4C and 0x5D respectively. Ensure that you create this program file in the folder having the name as your roll number. Give the screenshot.

Part4 – Using Constants

Q1. $P=Q+R+S$ with $Q=6$, $R=2$, $S=5$ and assume $R1=Q$, $R2=R$ and $R3=S$. We can use symbolic names Q , R and S using EQU (equate) assembler directive. The problem is solved using the assembly program shown below.

23CSE304: Embedded Systems

```
1;First assembly code example in Keil compiler.
2 AREA RESET, DATA, READONLY
3 EXPORT __Vectors
4 __Vectors
5 DCD 0x20001000 ; stack pointer value when stack is empty
6 DCD Reset_Handler ; reset vector
7
8 AREA MYCODE, CODE, READONLY
9 ENTRY
10 EXPORT Reset_Handler
11
12 Reset_Handler
13 MOV R0, #Q
14 MOV R1, #R
15 MOV R2, #S
16 ADD R3, R1, R0
17 ADD R4, R3, R2
18 LOOP B LOOP
19 Q EQU 6
20 R EQU 2
21 S EQU 5
22 END
```

Build Output

Program Size: Code=24 RO-data=8 RW-data=0 ZI-data=0
".\Objects\amma.axf" - 0 Error(s), 1 Warning(s).
Build Time Elapsed: 00:00:00

Execute this program which is saved in a folder renamed as your roll number. Give the screenshot of successful execution and answer the following questions. Which addressing mode is used in the first 3 assembly instructions in the program? _____. Give proper proof with the screenshot captured during the execution in the debug mode. What is the final value of R0? _____.

Part5 – Using Data in Memory

Q2. In the same problem $P=Q+R+S$ with $Q=6$, $R=2$, $S=5$ and assume $R1=Q$, $R2=R$ and $R3=S$. We will use load register LDR R1, Q instruction to load register R1 with the contents of memory location Q. This instruction does not exist and is not part of the ARM's instruction set. However, the ARM assembler automatically changes it into actual instruction.

Answer the following questions:

1. The code generated by ARM in the disassembly window for the pseudo instruction MOV R1, R is _____.
2. The value stored in the address 0x08000010 is _____.
3. The value of PC when it is executing the first instruction is 0x_____.
4. The address of R is _____, with the value of 2 in memory.
5. The address of S is _____, with the value of 5 in memory.

23CSE304: Embedded Systems

D:\Amritapuri\ES\2025-2026\CSE\Lab\STARCOM\Final Examples\STM32F446RE\amma.uvprojx - µVision [Non-Commercial Use License]

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help

Registers

Register	Value
Core	
R0	0x00000006
R1	0x00000002
R2	0x00000005
R3	0x00000008
R4	0x0000000D
R5	0x00000000
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x20001000
R14 (LR)	0xFFFFFFFF
R15 (PC)	0x0800001C
xPSR	0x01000000
Banked	
System	
Internal	
Mode	Thread
Privilege	Privileged
Stack	MSP
States	5

Disassembly

```
0x08000010 F04F0205 MOV r2,#0x05
16: ADD R3, R1, R0
0x08000014 EB010300 ADD r3,r1,r0
17: ADD R4, R3, R2
0x08000018 EB030402 ADD r4,r3,r2
18: LOOP B LOOP
0x0800001C E7FE B 0x0800001C
0x0800001E 0000 MOVS r0,r0
0x08000020 0000 MOVS r0,r0
0x08000022 0000 MOVS r0,r0
0x08000024 0000 MOVS r0,r0
0x08000026 0000 MOVS r0,r0
```

add.asm

```
11
12 Reset_Handler
13 MOV R0, #Q
14 MOV R1, #R
15 MOV R2, #S
16 ADD R3, R1, R0
17 ADD R4, R3, R2
18 LOOP B LOOP
19 Q EQU 6
20 R EQU 2
21 S EQU 5
22 END
```

Memory 1

Address	Value
0x08000000	00 10 00 20
0x08000004	09 00 00 08
0x08000008	4F F0 06 00
0x0800000C	4F F0 02 01
0x08000010	4F F0 05 02
0x08000014	01 EB 00 03
0x08000018	03 EB 02 04
0x0800001C	FE E7 00 00
0x08000020	00 00 00 00
0x08000024	00 00 00 00

Call Stack... Watch 1 Memory 1

Command